

HỆ THỐNG ĐẤU GIÁ TRỰC TUYẾN

Nhóm 15

Ngày 31 tháng 5 năm 2026

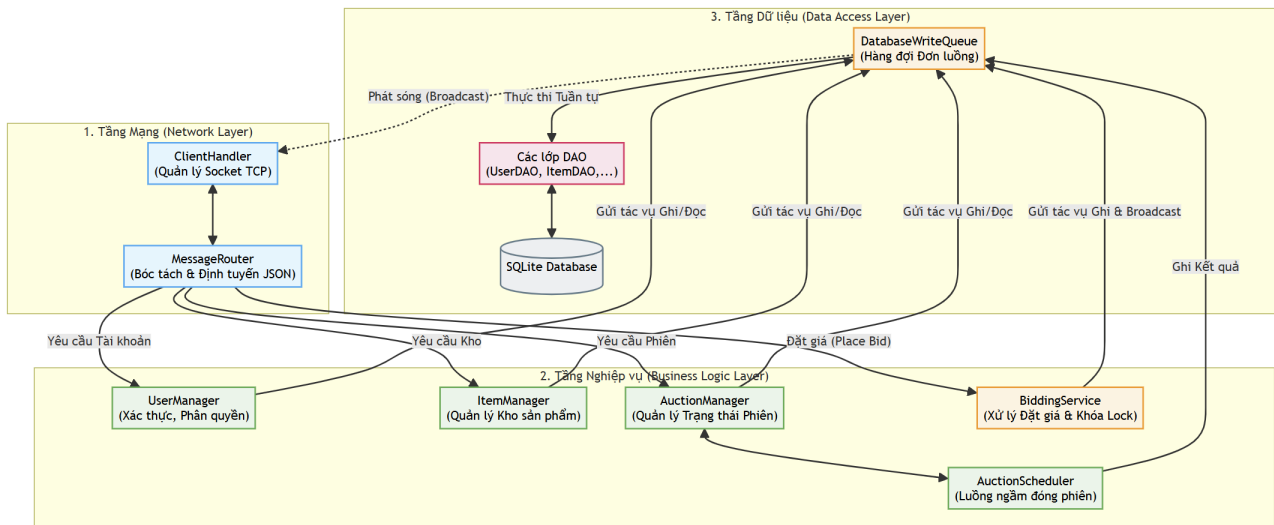
Mục lục

1	Tổng quan hệ thống	1
2	Thiết kế Hướng đối tượng & Design Pattern	2
3	Quản trị Cơ sở dữ liệu (SQLite)	3
3.1	Lý do lựa chọn	3
3.2	Cấu trúc dữ liệu và Tương tác	3
4	Xử lý Đa luồng & Đồng bộ hóa (Concurrency)	4
4.1	Bảo vệ dữ liệu trên RAM (ReentrantLock)	4
4.2	Bảo vệ dữ liệu Ổ cứng (Single-Threaded Write Queue)	4
5	Trực quan hóa Dữ liệu (Tính năng nâng cao)	4
5.1	Cấu trúc Biểu đồ	4
5.2	Giải pháp kỹ thuật và Tối ưu hóa UI	4
6	Kiểm thử Tự động (Unit Test)	5
7	Tích hợp & Phân phối liên tục (CI/CD)	5
8	Kết luận	5

1. Tổng quan hệ thống

- **Mô hình:** Client-Server, giao tiếp qua TCP Socket.
- **Kiến trúc phần mềm:** MVC (Model - View - Controller).
- **Định dạng dữ liệu mạng:** JSON.
- **Ngôn ngữ & Giao diện:** Java (JDK 26), JavaFX 21.0.1.
- **Phân quyền:** Admin (Quản trị), Seller (Người bán), Bidder (Người mua).

SƠ ĐỒ CẤU TRÚC CÁC LỚP XỬ LÝ LOGIC CHÍNH (SERVER)

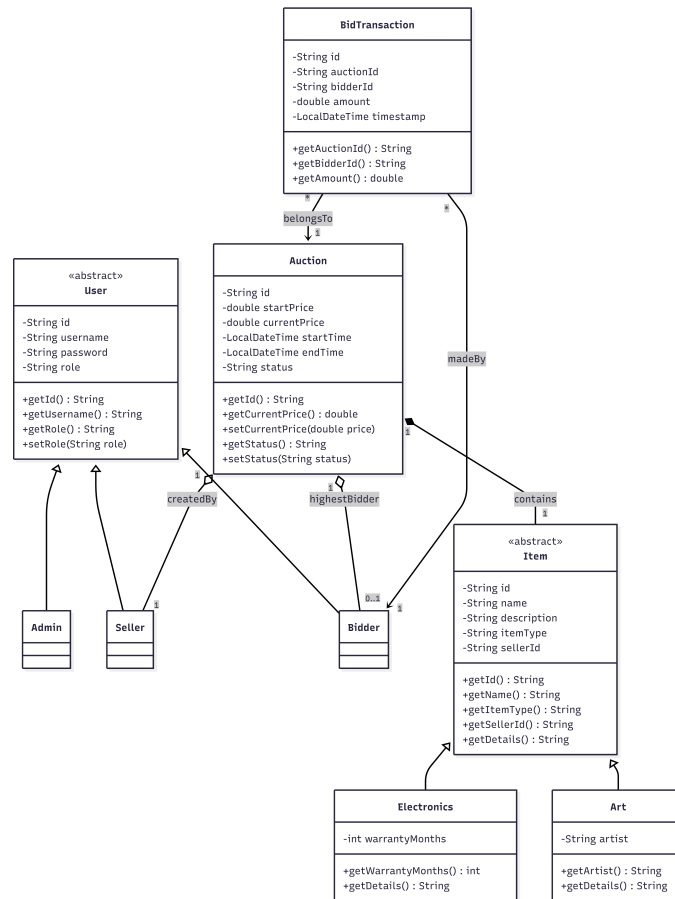


Chú thích luồng hoạt động:

- **Tầng 1:** Tiếp nhận gói tin từ Client, giải mã JSON và phân loại hành động.
- **Tầng 2:** Nơi chứa logic cốt lõi. Đặc biệt BiddingService sử dụng ReentrantLock để chặn Race Condition trên RAM.
- **Tầng 3:** Mọi tác vụ chạm vào CSDL hoặc Phát sóng mạng đều bị ép đi qua nút thắt DatabaseWriteQueue để đảm bảo xử lý đồng bộ, tuần tự (Strict Consistency).

2. Thiết kế Hướng đối tượng & Design Pattern

- **Tính Đa hình:** Ghi đè (Override) phương thức hiển thị chi tiết sản phẩm tùy theo loại (Electronics hiện bảo hành, Art hiện tên tác giả).
- **Tính Đóng gói:** Bảo vệ dữ liệu bằng private và các getter/setter.
- **Singleton Pattern:** Áp dụng cho các lớp quản lý tài nguyên tập trung.
 - DatabaseConnection
 - SceneManager
 - DatabaseWriteQueue
- **Factory Pattern:** Lớp ItemFactory giúp sinh đối tượng linh hoạt.



UML các lớp cơ bản

3. Quản trị Cơ sở dữ liệu (SQLite)

Hệ thống sử dụng **SQLite** để lưu trữ dữ liệu bền vững.

3.1. Lý do lựa chọn

- Nhẹ, không yêu cầu cài đặt máy chủ CSDL độc lập (Serverless).
- Tích hợp trực tiếp vào file .db, dễ dàng mang đi triển khai (Portable).

3.2. Cấu trúc dữ liệu và Tương tác

- **Thiết kế Schema:** Gồm 4 bảng chính: `users`, `items`, `auctions`, `bid_transactions` (chứa lịch sử đặt giá chi tiết).
- **Data Access Object (DAO):** Mọi thao tác truy vấn (SQL) được cô lập trong các lớp DAO (`UserDAO`, `ItemDAO`...), tách biệt hoàn toàn với logic nghiệp vụ.
- **Chống SQL Injection:** 100% các truy vấn đều sử dụng `PreparedStatement`.

4. Xử lý Đa luồng & Đồng bộ hóa (Concurrency)

Đây là giải pháp cốt lõi để giải quyết bài toán *Race Condition* (Tranh chấp dữ liệu) khi nhiều người đặt giá cùng một phần nghìn giây.

4.1. Bảo vệ dữ liệu trên RAM (ReentrantLock)

- Mỗi phiên đấu giá được cấp phát một ổ khóa `ReentrantLock` riêng biệt.
- Khi xử lý đặt giá (`BiddingService`), luồng mạng của Client A sẽ chiếm khóa. Luồng của Client B phải đứng chờ.
- **Kết quả:** Đảm bảo không bao giờ có 2 người cùng đặt thành công ở cùng một mức giá. Dữ liệu RAM luôn chính xác tuyệt đối.

4.2. Bảo vệ dữ liệu Ổ cứng (Single-Threaded Write Queue)

- **Vấn đề:** SQLite xử lý ghi đồng thời (Concurrent Writes) kém, dễ gây lỗi khóa database (Database Locked). Hơn nữa, việc đọc lịch sử ngay lúc đang ghi có thể gây ra hiện tượng *Read-After-Write Inconsistency*.
- **Giải pháp:** Toàn bộ lệnh GHI (Insert/Update), lệnh PHÁT SÓNG (Broadcast), và lệnh ĐỌC (Get History) được đẩy vào một `DatabaseWriteQueue`.
- **Cơ chế:** Hàng đợi này chỉ chạy **duy nhất 1 luồng** (Single-thread). Các tác vụ bị ép phải thực hiện nối đuôi nhau.
- **Kết quả:** Thứ tự hiển thị mạng khớp 100% với thứ tự lưu trong CSDL.

5. Trực quan hóa Dữ liệu (Tính năng nâng cao)

Giao diện phòng đấu giá được tích hợp biểu đồ **LineChart** của JavaFX để hiển thị diễn biến giá theo thời gian thực (Real-time Visualization).

5.1. Cấu trúc Biểu đồ

- **Trục X (CategoryAxis):** Hiển thị tên người dùng và số thứ tự lượt đặt (Ví dụ: `alice (#1)`, `bob (#2)`).
- **Trục Y (NumberAxis):** Hiển thị mức giá đặt (USD).

5.2. Giải pháp kỹ thuật và Tối ưu hóa UI

- **Bất đồng bộ UI:** Mọi gói tin TCP nhận được từ nền (Background thread) đều được gói trong `Platform.runLater()` trước khi vẽ lên biểu đồ, tránh lỗi *Not on FX application thread*.
- **Tránh tràn bộ nhớ đệm:** Khi tải lại phòng, hệ thống dọn dẹp nhãn cũ bằng `getCategories()` và `clear()`, khắc phục lỗi trôi nhãn (Label overlap) của JavaFX `CategoryAxis`.

6. Kiểm thử Tự động (Unit Test)

Dự án sử dụng **JUnit 5** với độ phủ (Coverage) cao, tập trung vào các logic chính của hệ thống. Các bộ test chính bao gồm:

- **BiddingServiceTest**: Giả lập nhiều Thread đặt giá cùng lúc. Kiểm tra độ chính xác của cơ chế **ReentrantLock**.
- **DatabaseWriteQueueTest**: Sinh 100 luồng ném tác vụ cùng lúc. Khẳng định hàng đợi xử lý đủ 100 tác vụ tuần tự mà không bị rớt dữ liệu (Thread-safe).
- **AuctionSchedulerTest**: Kiểm tra luồng đếm ngược, đảm bảo hàm đóng phiên tự động kích hoạt chính xác khi hết thời gian.
- **MessageRouterTest**: Dùng thư viện **Mockito** giả lập Client, kiểm tra tính đúng đắn của việc định tuyến JSON và các lệnh phân quyền Admin.

7. Tích hợp & Phân phối liên tục (CI/CD)

Dự án áp dụng quy trình CI/CD thông qua **GitHub Actions**.

- **Trigger**: Workflow tự động kích hoạt mỗi khi có mã nguồn được Push hoặc Pull Request lên nhánh main.
- **Kiểm định mã nguồn**: Máy chủ tự động thiết lập JDK, chạy toàn bộ bộ Unit Test bằng Maven. Nếu có Test Case thất bại, tiến trình Build sẽ bị chặn.
- **Đóng gói đa nền tảng (Fat JAR)**: Sử dụng **maven-shade-plugin** đóng gói toàn bộ thư viện phụ thuộc. Đặc biệt, cấu hình Maven tự động tải lỗi đồ họa Native (dll, so, dylib) của cả 3 hệ điều hành (Windows, Linux, macOS) nhét chung vào file **client.jar**.
- **Kết quả**: Người dùng cuối chỉ cần dùng lệnh **java -jar** để chạy hệ thống ở bất kỳ môi trường nào mà không gặp lỗi thiếu thư viện giao diện.

8. Kết luận

Hệ thống đã giải quyết trọn vẹn bài toán học búa nhất của mô hình Client-Server thời gian thực: **Đồng bộ hóa dữ liệu phân tán**. Việc thiết kế kiến trúc Hàng đợi (Queue), Khóa (Lock), bọc ngoại lệ đa tầng và tích hợp CI/CD tự động cho thấy ứng dụng đã đạt tiêu chuẩn của một phần mềm sẵn sàng triển khai thực tế (Production-ready).